أسئلة المناقشة: 100 سؤال متوقع وإجاباتها (بالمصري)

الملف ده للتدريب قبل المناقشة. كل سؤال ليه جزئين: **ببساطة** (الفكرة بكلام سهل، أحيانًا بمثال) و**الإجابة** (اللي تقوله للجنة). الأرقام والأسماء التقنية زي ما هي في المشروع، من غير تغيير.

تلات قواعد لكل إجابة

1. قول إن البيانات محاكاة (simulated) كل ما تذكر رقم.
2. فرّق بين اللي اتنفّذ فعلًا، واللي هو اختيار تصميم، واللي هو شغل مستقبلي.
3. لو مش عارف، قول اللي الـ repository بيوضّحه واعرض إنك تفتحه للجنة. ما تخمّنش رقم.

الأقسام: أ. البيزنس والنطاق (1-10) | ب. الـ Architecture واختيار التكنولوجيا (11-25) | ج. مصادر البيانات والـ simulator (26-33) | د. Kafka والـ streaming (34-42) | هـ. Bronze و Silver وجودة البيانات (43-55) | و. Gold و Databricks والـ attribution (56-66) | ز. Grafana والـ alerts والـ stock (67-78) | ح. Power BI والمقاييس (79-88) | ط. الاختبار والأداء والاعتمادية (89-95) | ي. الأمان والحدود والمستقبل والعملية (96-100)

أ. البيزنس والنطاق

1. إيه المشكلة اللي FunnelStream بيحلّها؟

ببساطة: الـ store بيصرف على ads، وأغلب orders بتاعته كاش عند الاستلام، وبيوعد بـ delivery سريع. بس مفيش مكان واحد يقوله الصورة كاملة.

الإجابة: الـ store مش قادر يجاوب في مكان واحد على تلات أسئلة: أنهي campaign بتجيب orders اتسلّمت فعلًا، وفين الـ customers بيسيبوا الـ funnel، وإيه اللي محتاج تصرف دلوقتي. FunnelStream بيربط الأحداث ورا التلات أسئلة في منصة واحدة، وليها عرضين: عرض لحظي للـ operations، وعرض تاني يوم الصبح للـ marketing.

2. مين المستخدمين؟

ببساطة: اتنين: واحد بيراقب دلوقتي، وواحد بيقرر الصبح.

الإجابة: مستخدم الـ operations بيتابع الترافيك اللحظي، والـ stock في 8 dark stores، و promise الـ delivery الـ express (20 دقيقة) في Grafana. ومستخدم الـ marketing بيقرا نتايج امبارح للـ campaigns والـ funnel والـ sales في Power BI.

3. إيه الحاجة الوحيدة اللي منصة مترابطة بس تقدر تعرضها؟

ببساطة: إنت بتصرف على ad بيودّي الناس لرف فاضي.

الإجابة: فلوس ads اتصرفت على products كانت خالصة من المخزن. الـ operations بتشوفها لحظيًا كطلب وصل لرف فاضي، والـ marketing بيشوفها تاني يوم الصبح كـ "exposed spend" مقدّر.

4. هل دي بيانات بيزنس حقيقية؟

ببساطة: لأ، كلها متولّدة بالـ simulator.

الإجابة: لأ. الـ customers والـ orders وأرقام الـ ads والـ stock كلها بيولّدها simulator بـ seed ثابت. الأرقام بتثبت إن الـ pipeline والطريقة شغالين، مش أداء بيزنس حقيقي، والـ dashboards بتقول كده صراحة.

5. ليه عملتوا محاكاة بدل dataset جاهزة؟

ببساطة: إحنا محتاجين بيانات كلها متّسقة مع بعض، وعايزين نحط فيها أخطاء معروفة ونعدّها.

الإجابة: كنا محتاجين stream events، و orders ليها دورة حياة، وبيانات ads، وأحداث stock، وكلهم متّسقين مع بعض، وكنا محتاجين نحقن أخطاء معروفة ونعدّها. أي dataset عامة مش بتدّي stream متّسق فيه stock و delivery express وربط بالـ campaigns. والـ seed بيخلّي كل demo واختبار replay قابل للتكرار.

6. المحاكاة واقعية قد إيه؟

ببساطة: عملناها على شكل store مصري حقيقي، بس مش هنقول إنها مطابقة 100%.

الإجابة: بتحاكي store مصري: أسماء وأرقام تليفونات مصرية، ومدن موزّعة حسب عدد السكان بمناطقها الحقيقية، والجنيه EGP، والكاش عند الاستلام كأكبر وسيلة دفع، وفوري وفودافون كاش وإنستاباي، وشركات الشحن Bosta و Aramex و J&T، وذروة رمضان والعيد. مش نقدر نثبت إنها واقعية بالكامل، لكن الافتراضات متوثّقة والتوزيعات متراجعة عليها.

7. أنهي قرارات بيزنس النظام بيدعمها؟

ببساطة: قرارات صغيرة وواضحة: فلوس الـ ad، الـ funnel، الـ stock، الـ delivery.

الإجابة: نقل الـ budget بين الـ campaigns بناءً على ROAS للـdelivered orders، وإصلاح الخطوة والـ channel اللي فيهم أعلى abandonment، وإعادة تخزين الـ products اللي خلصت، ومتابعة orders الـ express اللي هتفوّت وعدها، ووقف ads الـ products اللي خلصت. وكل ده معروض على بيانات محاكاة.

8. إيه نطاق المشروع، وإيه اللي سبتوه قصدًا؟

الإجابة: جوه النطاق: الـ ingestion، والمعالجة الـ streaming، وفحوصات الجودة، ونموذج lakehouse، ومراقبة لحظية بـ alerts، وتقرير BI من 7 صفحات، ومساعد LLM بيجاوب على الأسئلة. اللي سبناه قصدًا: customers حقيقيين، ونشر Kafka و Spark على السحابة، والتوصيات (جدول `rec_product` فاضي كمكان محجوز)، واختبار incrementality، و contribution margin.

9. ليه أداتين: Grafana و Power BI؟

ببساطة: ده زي عداد السرعة في العربية مقابل تقرير الصيانة الشهري. واحد للحظة، والتاني للقرار.

الإجابة: هما بيخدموا مستخدمين بسرعات مختلفة. Grafana بيقرا جداول Postgres صغيرة بتتحدّث في أقل من دقيقة وفيه alert rules، فمناسب للـ operations. Power BI بيقرا جداول Gold بعد المطابقة والـ daily marts، فمناسب للـ marketing اللي محتاج أرقام موثوقة لامبارح. Grafana بيجاوب "السيستم سليم وإيه محتاج تصرف دلوقتي"، و Power BI بيجاوب "إيه اللي حصل وإيه نقرر".

10. إيه أهم نتيجة؟

الإجابة: pipeline streaming بيستحمل crash من غير تكرار، وبيانات وحشة بتتحجز في quarantine بسببها وبتتعدّ مقابل الـ noise اللي اتحقن، وجمهورين بيخدموا من نفس الأحداث، وجسر واحد بينهم. الأدلة المقاسة: 66 data quality check من غير ولا فشل، و 1,169 اختبار على الـ host نجحوا، و `NO DUPLICATES` في تجربتين kill-and-restart.

ب. الـ Architecture واختيار التكنولوجيا

11. اشرح الـ architecture في دقيقة.

ببساطة: البيانات بتدخل من الشمال، بتتنضّف في النص، وبتتقسم لمسارين.

الإجابة: simulator و mock ad API بيولّدوا events. Kafka بيشيل 4 topics. وSpark Structured Streaming جوه Docker بيكتب Bronze (الخام) و Silver (متحقق منه وملغي التكرار) بصيغة Delta، ومعاهم جدول quarantine للصفوف المرفوضة. من Silver، المسار اللحظي بيكتب جداول Postgres صغيرة لـ Grafana و 8 alert rules. والمسار الصباحي بينشر Silver على Databricks، وبيبني Gold على star schema بـ 66 check، وبيضيف daily marts في Postgres، وبيغذّي Power BI.

12. ليه تصميم medallion (Bronze, Silver, Gold)؟

ببساطة: زي مصنع: مخزن مواد خام، بعدين خط تنضيف، بعدين product جاهز للبيع.

الإجابة: كل layer ليها شغلة واحدة. Bronze بيحتفظ بكل رسالة خام للأبد، فمفيش حاجة بتضيع وممكن نعيد البناء. Silver هو النسخة الموثوقة الوحيدة: typed ومتحقق منه وملغي التكرار. Gold جاهز للبيزنس: facts و dimensions و aggregates. فصلهم بيخلّينا نصلّح قاعدة في Silver أو Gold ونعيد البناء من غير ما نرجع للمصدر.

13. ليه مسارين بدل pipeline واحدة؟

ببساطة: حد عايز الرقم بسرعة حتى لو تقريبي، وحد عايز الرقم مظبوط حتى لو اتأخر.

الإجابة: القرارات محتاجة سرعات مختلفة ودرجات تأكّد مختلفة. الـ operations محتاجة رقم في دقيقة حتى لو event متأخر ممكن يغيّره. والـ marketing محتاج رقم متطابق ومبيتحرّكش. المسار اللحظي بيضحّي بالتأكّد عشان السرعة وبيسيب الـ late events بره. والمسار الصباحي بيحطهم ويشغّل 66 check.

14. ليه Kafka؟

ببساطة: زي صندوق بريد: اللي بيبعت مش لازم يستنى اللي بيستلم.

الإجابة: بيفصل المنتجين عن المعالجة، وبيحتفظ بالرسايل لما Spark job يكون واقف، وبيسمح بـ replay بعد أي crash. الـ keys بتحافظ على ترتيب كل session وكل order. وKafka بيشتغل محليًا في container واحد، وده مناسب لـ deployment على لاب توب. ما جرّبناش brokers تانية، فالادعاء الصادق إن Kafka مناسب للاحتياج، مش إنه فاز على بدائل.

15. ليه Spark Structured Streaming و Delta Lake؟

الإجابة: بيدّونا checkpoints وجداول partitioned و `MERGE` Delta، ومع بعض بيخلّوا الكتابة idempotent (لو اتكررت مفيش أثر)، ونفس الكود يقدر يعيد تشغيل Bronze لإعادة بناء Silver. ما قيّمناش Flink أو ksqlDB في المشروع ده، فمش بنقارن.

16. ليه Spark بيشتغل في Docker؟

الإجابة: على Windows، Spark محتاج winutils وبيئة شبه Linux، فالـ Spark jobs واختباراتها بتشتغل في container. Java و Spark جوه الـ Docker image ومش متسطّبين على الـ host.

17. ليه Databricks لـ Gold؟

الإجابة: بيدّينا SQL warehouse و Unity Catalog للـ star schema، وبيوضّح الجزء السحابي في التصميم: الـ lake المحلي فيه Bronze و Silver، و Databricks فيه نسخة Silver و Gold. ده workspace من Free Edition، فليه حدود، والـ benchmark بتاعنا على setup واحد، مش ادعاء سعة.

18. ليه Postgres للجداول اللحظية؟

ببساطة: Grafana بيسأل كل كام ثانية، فمحتاج جداول صغيرة وسريعة.

الإجابة: Grafana محتاج جداول صغيرة وسريعة ومفهرسة يقدر يسألها كل كام ثانية. الـ live sink بيعيد حساب الدقايق وأزواج الـ store والـ product اللي الـ batch لمسها بس، وبيكتبها في Postgres. وPostgres كمان فيه الـ daily marts اللي Power BI بيقراها. هو مش المخزن التحليلي.

19. ليه Grafana للمراقبة؟

الإجابة: بيعمل provisioning للـ dashboards والـ alert rules كـ code، وبيدعم data source read-only، وبيتعامل كويس مع الفجوات والـ thresholds. 6 dashboards و 8 alert rules بتتولّد من Python builders، وفي اختبار بيفشل لو أي ملف ابتعد عن الـ builder بتاعه.

20. ليه Power BI للطبقة البيزنس؟

الإجابة: هو الأداة المعروفة للبيزنس، وبيدعم semantic model بـ measures و relationships، وصيغة المشروع (`.pbip`) بتخلّي التقرير ملفات نصية generator يقدر يطلّعها واختبار يقدر يفحصها. استخدمنا Import mode بـ credentials للقراءة فقط.

21. إيه دور Airflow؟

الإجابة: DAG واحد اسمه `ads_yesterday_report` بيشتغل UTC 03:00، بيسحب من ad API مرة واحدة لليوم اللي خلص، وبيخزّن snapshot وبيكتب تقرير JSON. بيعيد المحاولة لو جه HTTP 503، وبيرجّع التقرير المخزّن لو اتشغّل تاني. هو optional Docker profile، ونفس الخطوات بتشتغل كـ Python script.

22. ليه بيانات Airflow بتروح Silver مباشرة مش Bronze؟

ببساطة: Bronze مخصّص لرسايل Kafka الخام، وتقرير الـ ads مش جاي من Kafka.

الإجابة: Bronze فيه رسايل Kafka الخام بـ partition و offset و وقت الـ ingest. تقرير الـ ads مش جاي من Kafka. هو سحب HTTP واحد في اليوم، بيتحقق منه وقت السحب وبيتخزّن في `silver_campaign_snapshots`. كل سحبة بتتحفظ كـ snapshot لوحدها بمفتاح (الـ campaign، تاريخ التقرير، وقت السحب)، والصفوف الوحشة بتروح quarantine. المقابل إن الـ response الخام مش محفوظ. جدول `bronze_ad_pulls` هيقفل الفجوة دي، وده الخطوة الجاية الصادقة.

23. هل الـ architecture دي نفسها في production؟

الإجابة: لأ. كل اللي على شمال Databricks بيشتغل على لاب توب Windows واحد في Docker. في production هنستخدم Kafka مُدار، و object storage للـ lake، و cluster لـ Spark، و secrets manager، و CI، ومراقبة للمنصة نفسها. دي مقترحات مش features متنفّذة.

24. إيه مبادئ التصميم؟

الإجابة: طبقات medallion، وتسليم at-least-once مع كتابة idempotent، وكاتب واحد لكل جدول، ومسارين، وعقود عند الباب (schema validator)، وتعريفات مقاييس معلنة في قاموس واحد، وبدائل زي store Postgres ورا نفس الـ interface بتاع Databricks.

25. يعني إيه "كاتب واحد لكل جدول" وليه مهم؟

ببساطة: لو شخصين بيكتبوا في نفس الدفتر هيحصل تعارض، ولو الرقم غلط مش هنعرف مين السبب.

الإجابة: كل جدول Gold بيكتبه الـ Databricks Gold build بس، والـ Spark stream المحلي بيكتب Bronze و Silver والجداول اللحظية بس. كاتب واحد لكل جدول بيمنع تعارض الكتابة وبيوضّح الرقم الغلط جه منين.

ج. مصادر البيانات والـ simulator

26. إيه الأحداث اللي الـ simulator بيطلّعها؟

الإجابة: أحداث التطبيق والويب مع تساقط الـ funnel (`app_open` , `page_view` , `product_view` , `product_click` , `product_search` , `add_to_cart` , `remove_from_cart` , `checkout_started` , `purchase`)، و orders وتوصيلات بدورة حياة كاملة، و snapshots وحركات stock لـ 8 dark stores، وأرقام campaigns من mock Meta و Google Ads API. الكتالوج فيه حوالي 200 product في 4 أقسام و 5,000 customer.

27. إيه اللي جوه الـ event؟

ببساطة: كل event ليه "ظرف" ثابت، زي الخطاب فيه عنوان وتاريخ ومحتوى.

الإجابة: كل event بيستخدم envelope واحد: `event_id` و `event_type` و `event_version` و `event_time` وكائن `payload` . الأحداث المجهولة ملهاش `customer_id` وبتتـ key بـ `session_id` .

28. ليه بتحقنوا noise قصدًا؟

ببساطة: لو الـ stream نضيف 100%، مفيش حاجة تثبت إن الفحص شغال. زي ما تحط ورقة غلط في امتحان عشان تتأكد إن المصحّح هيلقطها.

الإجابة: الـ stream النضيف مش بيثبت حاجة عن جودة البيانات. الـ simulator بيحقن تكرارات (1%)، و JSON مكسور (0.3%)، وحقول ناقصة، وأنواع مش معروفة، وأسعار سالبة، وتواريخ في المستقبل، و late events (2% زائد 0.3% متأخرة جدًا)، ونصوص وحشة، و bots، وحالات order بترتيب غلط، وصفوف ads وحشة. وبيكتب العدد في `noise_manifest.json` ، فنقدر نتأكد إن الـ pipeline مسك اللي اتحقن.

29. عرفتوا منين إن الـ pipeline مسك كل الـ noise؟

الإجابة: بعد Silver، عدد الـ quarantine بيتقارن بالـ noise manifest ولازم يتطابق في حدود تسامح. في تشغيل 13 أكتوبر على الـ scratch، كان متوقع 308 صف واتلاقوا 308.

30. يعني إيه "seeded"؟

ببساطة: زي ما ترمي نرد بنفس الرمية كل مرة: نفس النتيجة.

الإجابة: نفس الـ seed بيدّي نفس الـ customers والـ products والأحداث. ده بيخلّي الـ demo قابل للتكرار وبيسمح لاختبارات الـ replay بمقارنة النتايج. سكريبت الـ demo بيختار seed جديد كل مرة عشان الـ ids ما تتكررش.

31. هل الـ ad API حقيقي؟

الإجابة: لأ، هو mock لـ API تقارير Meta و Google Ads. أرقامه متّسقة مع قيم `utm_campaign` في الترافيك المتولّد. وبيقدر يفشل قصدًا (HTTP 503، سحبات مكررة، spend ، `clicks > impressions` سالب) عشان نختبر الـ DAG والـ validator.

32. ليه 8 dark stores؟

الإجابة: بيمثّلوا حالة الـ quick-commerce في مناطق مصرية حقيقية: مدينة نصر والمعادي والقاهرة الجديدة في القاهرة، والدقي في الجيزة، والمنتزه ورمل في إسكندرية، وواحد في المنصورة وواحد في طنطا. كل منطقة ليها store أساسي واحد. أسيوط و"أخرى" ملهمش store ومستبعدين من مقاييس stock-out.

33. هل الـ simulator بيحاكي الـ stock بواقعية؟

الإجابة: فيه `StockLedger` بيمنع الـ stock من النزول تحت الصفر، وبيطلّع snapshot افتتاحي لكل زوج store و product، و snapshot أساسي كل 5 دقايق، وحركة `sold` لكل سطر order. أحداث الطلب مش بتتوقف لما الـ stock يفضى، لأن الطلب الضايع ده هو اللي بنقيسه. حد معروف: الـ stock بيتحجز لما الـ order يتعمل ومش بيرجع عند الـ cancellation أو الإرجاع أو فشل الـ delivery.

د. Kafka والـ streaming

34. إيه الـ topics وليه الأربعة دول؟

ببساطة: قسّمناهم حسب نوع الحدث، وكل مفتاح بيحافظ على ترتيب أحداثه.

الإجابة: `session_id` key، 6 partitions) `app.events`)، و `order_id` key، 3) `orders`)، و `delivery.events` key، 3) `store_id` key، 3) `inventory.events`). اتقسّموا حسب الكيان وحسب احتياج الترتيب، فكل key بيحافظ على ترتيب أحداثه جوه partition واحد. `app.events` ليه 6 partitions لأنه أكتر stream زحمة بفارق كبير، والتلاتة التانيين 3 لأنهم بيشيلوا جزء صغير من الترافيك ده. الـ repository بيسجّل الأرقام بس من غير سبب مكتوب، فقول إنها اتختارت بما يناسب الترافيك وحجم لاب توب واحد (الـ Spark job بيشتغل بـ `local[4]` ، أربع tasks متوازية)، وإننا ما قسناش قيم تانية. في production العدد هيتحدد من أقصى throughput وعدد الـ consumer tasks مع هامش، لأن عدد الـ partitions ممكن يتزوّد بعدين لكن مينفعش يتقلّل.

35. ليه الـ keys دي؟

الإجابة: كل أحداث الـ session وكل أحداث الـ order بتروح partition واحد بالتسلسل. ده بيحافظ على ترتيب كل session وكل order، واللي منطق دورة حياة الـ order معتمد عليه.

36. إيه ضمان التسليم عندكم؟

ببساطة: ممكن الرسالة توصل مرتين، بس لو وصلت مرتين مفيش أي أثر. النتيجة نفسها كأنها وصلت مرة واحدة.

الإجابة: at-least-once مع كتابة idempotent. منتجو Kafka بيستخدموا `acks=all` و idempotence، و Silver بيشيل التكرار على `event_id`، و `MERGE` Delta على الـ key معناه إن الرسالة المعادة مش بتضيف حاجة. النتيجة exactly-once في الأثر، من غير إعداد exactly-once على مستوى الـ broker.

37. إيه اللي بيحصل لو Spark job عمل crash؟

ببساطة: زي ما تعمل bookmark في كتاب: ترجع لنفس الصفحة.

الإجابة: الـ checkpoints على الـ disk فيها الـ Kafka offsets اللي اتـ commit. لما الـ job يشتغل تاني بنفس الأمر ونفس فولدر الـ checkpoint، بيكمّل من الـ offsets دي وبيقرا اللي Kafka احتفظ بيه وهو واقف.

38. جرّبتوا ده؟

الإجابة: أيوه. في الـ failure demo قتلنا Silver job بـ `docker kill`، وشفنا alert "Silver job silent" بيطلع بعد 2 دقيقة و 12 ثانية و 2 دقيقة و 54 ثانية من القتل في تجربتين، وعملنا restart بنفس الـ checkpoints، وفحص التكرار طلع `NO DUPLICATES` في المرتين. الفحص بيغطي Silver بس، ومبيثبتش إن مفيش event ضاع.

39. إيه الـ micro-batch trigger؟

الإجابة: عشر ثواني. على اللاب توب ده، batches الـ Silver ممكن تكون أبطأ من الـ trigger و Spark بيطلّع تحذير "batch is falling behind". ده طبيعي هنا، وإحنا بنقفل Gold sync أثناء الـ demo.

40. إزاي بتتعاملوا مع وقت الـ event؟

الإجابة: الأحداث بتشيل `event_time` بتوقيت UTC. الـ event بيعتبر متأخر لما وقت Kafka ناقص وقت الـ event يزيد عن 10 دقايق. الـ late events بتفضل في Silver و Gold، وبتتعلّم `is_late`، وبتتسيب بره المقاييس اللحظية، وبتتصحّح في الـ batch. أيام البيزنس بتتحسب بتوقيت `Africa/Cairo`.

41. إزاي بتتعاملوا مع حالات الـ order اللي جت بترتيب غلط؟

ببساطة: مش بنثق في ترتيب الوصول. بنثق في أعلى "مرحلة" وصل لها الـ order.

الإجابة: الحالة الحالية للـ order هي الحالة صاحبة أعلى rank: `order_created` 1، `payment_authorised` 2، `out_for_delivery` 4، `order_shipped` 3، `order_packed` و `order_completed` 6، `delivered` 5، `delivery_failed` و `order_cancelled` 7. `order_returned` وهي نهائية، والـ cancellation بعد الشحن بيترفض لـ quarantine.

42. ليه مش بنقرا من Kafka مباشرة لـ Postgres من غير Bronze و Silver؟

الإجابة: هنخسر النسخة الخام، و cancellation التكرار، والـ quarantine، والـ replay، وجداول Silver اللي بتغذّي Gold. Bronze و Silver هما اللي بيخلّوا الاسترجاع من crash وإعادة البناء وإثبات الجودة ممكنين.

هـ. Bronze و Silver وجودة البيانات

43. إيه اللي في Bronze؟

الإجابة: قيمة الرسالة الخام زائد Kafka partition و offset و timestamp ووقت الـ ingest. هو append-only ومفيش حاجة بتتحذف أو تتغيّر. هو المصدر لأي إعادة بناء.

44. Silver بيعمل إيه؟

ببساطة: بياخد الخام وبيعمل منه نسخة نضيفة وموثوقة.

الإجابة: بيتحقق من schema ليه version، وبيحوّل الحقول لأنواعها، وبيوحّد القيم (UTC، نص المدينة والجهاز، قيم UTM)، وبيشيل التكرار على الـ key، وبيكتب بـ `MERGE` Delta. كل جدول Silver ليه key، فإعادة تشغيل batch مبتغيّرش حاجة. والـ batch بيتشال منه التكرار قبل الـ `MERGE` ، فصفّين بنفس الـ key مبيتقابلوش في merge واحد.

45. إيه اللي بيروح quarantine؟

الإجابة: الصفوف اللي بتفشل في التحقق، مع كود الخطأ والـ payload الخام: `BAD_JSON` و `MISSING_FIELD` و `UNKNOWN_EVENT_TYPE` و `BAD_VERSION` و `NEGATIVE_PRICE` و `FUTURE_TIMESTAMP` و `UNKNOWN_PRODUCT` ، وللـ ads `NEGATIVE_SPEND` و `CLICKS_GT_IMPRESSIONS` . الصف المحجوز عمره ما بيوصل للأرقام.

46. ليه quarantine بدل ما نحذف الصفوف الوحشة أو نصلّحها؟

ببساطة: لو رميت الورقة الغلط مش هتعرف مين غلط. ولو صلّحتها بالتخمين، هتألّف بيانات.

الإجابة: الحذف بيخبّي مشكلة الـ product. والتخمين بيخترع بيانات. الـ quarantine بيحتفظ بالدليل، وبيدّي سبب بيشاور على الـ producer اللي لازم يتصلّح، وبيخلّينا نعدّ الصفوف المرفوضة مقابل الـ noise المحقون. ونسبة الـ quarantine كمان بتشغّل alert عند 2%.

47. هل Silver ممكن يتعاد بناؤه؟

الإجابة: أيوه، من Bronze في أي وقت، لكل اللي بيعدّي من Kafka. الاستثناء الوحيد هو snapshots الـ ads، لأنها مبتعدّيش من Bronze. عملنا replay لـ Bronze في الـ smoke test و 10 جداول Silver ما اتغيّرتش.

48. إزاي بتشيلوا التكرار؟

الإجابة: الـ simulator بيكرّر 1% من الأحداث بنفس الـ Silver . `event_id` بيشيل التكرار على `event_id` جوه الـ batch، والـ `MERGE` بيتجاهل أي key موجود. و `scripts/check_duplicates.py` بيقرا كل جدول Silver ليه key وبيبلّغ عن أي تكرار.

49. إيه جداول Silver؟

الإجابة: `silver_events` و `silver_orders` و `silver_order_status` و `silver_order_items` و `silver_stock_movements` و `silver_stock_snapshots` و `silver_express_orders` و `silver_customers` و `silver_products` و `silver_quarantine` ، وزيهم `silver_campaign_snapshots` من الـ ad API.

50. إيه هي الـ bot sessions وبتعملوا فيها إيه؟

الإجابة: حوالي 0.1% من الـ sessions بتبقى bots فيها مئات الأحداث في ثواني. بتاخد علامة `is_bot` وبتتسيب بره الـ funnel والـ attribution والمقاييس اللحظية. حد معروف: مسار demand الـ out-of-stock products مش بيشيل الـ bot sessions،

لكن Gold بيشيلها.

51. إزاي بتتحققوا من عقود البيانات (data contracts)؟

الإجابة: validator في `contracts/` بيفحص كل event على schema ليه version حسب نوعه. الـ event الوحش بيروح quarantine بكود خطأ. الـ validator والـ schemas مغطيين باختبارات.

52. إيه هي الـ data quality check 66؟

ببساطة: بعد كل Gold build بنشغّل 66 فحص. 46 منهم لو لقوا مخالفة بيوقّفوا الـ batch. و 20 بيطبعوا رقم بس.

الإجابة: 66 نتيجة في كل Gold build: 46 فحص من نوع fail بيوقّفوا الـ batch لو لقوا مخالفة، و 20 فحص من نوع report بيطبعوا رقم بس (نسب وأعداد صفوف). المجموعات: تكرار المفاتيح، ونضافة Silver (مفيش `event_id` فاضي، ومفيش حاجة في المستقبل، ومفيش product مجهول، و `line_total = quantity x unit_price`)، والـ hierarchy والتاريخ (صف حالي واحد لكل key، ومفيش فترات صلاحية متداخلة، والمنطقة تابعة لمدينتها)، والمطابقة (مجموع الـ orders بيساوي سطورها، وعدد Gold بيطابق Silver)، وفحص الـ manifest (الـ quarantine في حدود 2% من الـ manifest)، والنسب المُبلّغ عنها.

53. الـ manifest check بيثبت إيه؟

ببساطة: إحنا عارفين بالظبط حطّينا كام غلطة، فنقدر نتأكد إن الـ pipeline لقاهم كلهم ومارمّاش حاجة سليمة.

الإجابة: إن عدد الـ quarantine عند الـ pipeline بيساوي الأخطاء اللي الـ simulator بيقول إنه حقنها، في حدود 2%. ده دليل إن الـ validator مش بيفوّت أخطاء ومش بيرفض صفوف سليمة. وبيشتغل لأن الـ simulator بتاعنا، فعارفين العدد الحقيقي.

54. إيه اللي بيحصل للـ late events؟

الإجابة: بتتحفظ. بتتعلّم، وبتتسيب بره المقاييس اللحظية عشان المسار اللحظي ميتحرّكش بعد الواقعة، وبتتحط في الـ batch عشان Gold و Power BI يبقوا صح.

55. إزاي بتتعاملوا مع البيانات الشخصية؟

الإجابة: الأسماء وأرقام التليفونات موجودة في Bronze و Silver بس. النشر على Databricks بيستخدم allow-list للأعمدة، فمبتوصلش لنسخة Silver ولا Gold ولا Power BI. Gold والتقرير فيهم `customer_id` بس. وكل البيانات اصطناعية.

و. Gold و Databricks والـ attribution

56. إيه اللي في Gold؟

الإجابة: اتناشر جدول: `dim_customer` و `dim_product` (slowly changing من النوع 2)، و `dim_campaign` و `dim_date` و `fact_orders` و `fact_order_lines` و `fact_attribution` و `agg_orders_daily` و `agg_funnel_daily` و `agg_campaign_daily` و `agg_city_product_daily` و `customer_360`. وفي مكان تالتاشر محجوز اسمه `rec_product` فاضي قصدًا.

57. إيه هو SCD2 وفين بتستخدموه؟

ببساطة: لما حاجة تتغيّر (سعر product مثلًا)، مش بنمسح القديم. بنضيف صف جديد ونفضل نعرف إمتى كان كل سعر ساري.

الإجابة: الـ slowly changing dimension نوع 2 بيحتفظ بالتاريخ بإضافة صف جديد لكل تغيير، بـ `valid_from` و `valid_to` و `dim_customer` . `is_current` بيتتبّع المدينة والمنطقة والجهاز الأساسي. `dim_product` بيتتبّع القسم والسعر وحالة النشاط. الـ simulator بيغيّر الأسعار والأقسام قصدًا عشان التاريخ يتكوّن. وفي فحوصات بتتأكد من صف حالي واحد لكل key ومفيش فترات صلاحية متداخلة.

58. إزاي الـ facts بتلاقي صف الـ dimension الصح؟

الإجابة: مفاتيح الـ dimension بتتحل وقت بناء الـ fact، من نسخة الـ dimension اللي كانت سارية وقت الـ event. الصف اللي ملوش تطابق بيروح quarantine ومش بياخد key مخمّن. المفتاح -1 هو العضو المجهول في الـ dimensions.

59. إزاي بتنشروا Silver على Databricks؟

ببساطة: Databricks مش شايف الـ Kafka اللي على اللاب توب، فبنبعت له ملفات.

الإجابة: Databricks مش بيقدر يوصل لـ Kafka اللي على اللاب توب، فـ Silver بيعدّي كملفات.
`scripts/publish_silver.py` بيصدّر Silver بـ allow-list للأعمدة، وبيرفع Parquet على Unity Catalog volume، وبيعمل `MERGE` في نسخة Silver. هو insert-only و idempotent. وفي خيار `--reset` اختياري بيفرّغ جداول النسخة قبل نشر جديد.

60. إزاي الـ attribution بيشتغل؟

ببساطة: السؤال "الـ order ده نسبه لأنهي campaign؟". عندنا طريقتين: آخر ضغطة، وأول ضغطة.

الإجابة: فيه نموذجين. `last_touch` بينسب الـ order لـ campaign الـ click بتاع الـ session نفسه أولًا، ولو مفيش فآخر click للـ customer في الـ 7 أيام اللي قبله. `first_touch` بينسبه لأول click. الـ order اللي مفيش وراه click بيعتبر direct بـ campaign NULL.

61. ليه الـ ROAS الأساسي على الـ delivered orders؟

ببساطة: الـ order اللي اتسجّل واتلغى مجابش فلوس. فالأمين إننا نحسب على اللي اتسلّم.

الإجابة: orders كتير كاش عند الاستلام وبتتلغي أو بيفشل توصيلها، فـ ROAS على الـ placed orders بيبالغ في العائد. الرقم الأساسي هو `revenue_delivered / spend` ، والـ ROAS على الـ placed ثانوي. التسليم بيتأخر عن الـ spend من يومين لأربعة، فالأيام الأخيرة بتبان واطية وبتتعلّم provisional.

62. هل الـ attribution بيثبت إن الـ campaign هي اللي سبّبت الـ order؟

الإجابة: لأ. last touch بيفضّل القنوات اللي في آخر الـ funnel زي البحث و retargeting، ومفيش نموذج بيثبت incrementality. التقرير بيعرض النموذجين. واختبارات الـ incrementality مكتوبة كشغل مستقبلي.

63. إيه الفرق بين CPA وتكلفة الـ delivered order؟

الإجابة: CPA هو الـ spend مقسوم على الـ placed orders. وتكلفة الـ delivered order هي الـ spend مقسوم على الـ delivered orders بس. هما بيختلفوا بالتعريف. مثال: في صفحة الـ `ramadan_offer_1` ،campaigns فيها CPA بـ 171 وتكلفة الـ delivered order بـ 377.

64. ليه أرقام الـ ads على يوم مختلف عن الـ orders؟

الإجابة: منصة الـ ads بتبلّغ بيوم UTC، والـ orders بتنتمي ليوم القاهرة. الفرق ما بيزيدش عن 3 ساعات، والاختلاف ده مقبول ومكتوب في الصفحات اللي بيأثّر فيها.

65. إيه البديل في Postgres لـ Gold؟

الإجابة: فيه interface اسمه `GoldStore` ليه backend لـ Databricks و backend لـ Postgres، فجداول Gold تقدر تتحمّل في Postgres لو Databricks مش متاح. وخطوات Airflow كمان بتشتغل كـ Python script.

66. إيه اللي طلّعه الـ Gold build في آخر تشغيل؟

الإجابة: على الـ scratch catalog يوم 13 أكتوبر، الـ build خلص بـ `BATCH OK` في 175.6 ثانية بـ 66 check وصفر فشل، وعدد الـ quarantine طابق الـ manifest (308 متوقع، 308 اتلاقوا).

ز. Grafana والـ alerts والـ stock

67. عندكم كام dashboard وكام alert؟

الإجابة: 6 dashboards: الـ pipeline اللحظي، و stock الـ operations اللحظي، و express SLA، و live stock monitor، وتحليلات الـ stock اليومية، و D-1 marketing. و 8 alert rules: نسبة الـ late events، ونسبة الـ quarantine، و Silver job silent، ولا أحداث، وطلب على الـ out-of-stock products، و order express فوّت وعده، ونسبة الـ express on-time تحت الهدف، و products كتير خالصة في store واحد.

68. إزاي الـ dashboards والـ alerts بتتدار؟

ببساطة: هي ملفات كود مش حاجة بتتعمل بالإيد في الواجهة.

الإجابة: كـ Python builders. code في `monitoring/` بتولّد الـ dashboard JSON والـ alert rules، و Grafana بيعمل provisioning من الملفات، واختبار بيفشل لو أي ملف ابتعد عن الـ builder. بعد أي تعديل في dashboard لازم نعمل restart لـ Grafana container.

69. الـ alert "Silver job silent" بيعمل إيه؟

الإجابة: بيشتغل لما الـ Silver heartbeat يبقى أقدم من دقيقتين، وكمان لو مفيش بيانات خالص. في الـ failure demo اشتغل بعد 2 دقيقة و 12 ثانية و 2 دقيقة و 54 ثانية من القتل، فالـ promise اللي بنقوله هو "في خلال 3 دقايق".

70. إزاي بتعرّفوا stock-out (out of stock)؟

الإجابة: الـ product خالص لما `available <= 0`، وقليل لما `0 < available < reorder point`. الـ stock محاكى، والجداول اللحظية بتعرض الحالة الحالية و snapshot كل دقيقة لكل store بيتحفظ 24 ساعة.

71. إزاي بتقيسوا promise الـ express؟

ببساطة: كل order express ليه promise بالدقايق (20 في production). لو وصل في الوقت، فـ on time. لو اتأخر، فـ breached.

الإجابة: orders الـ express بتشيل `promised_minutes` بتاعتها (20 في production). On time يعني اتسلّم في خلال الـ promise. Breached يعني اتسلّم متأخر، أو لسه مش متسلّم بعد الـ promise زائد دقيقة سماح. At risk يعني مش متسلّم وخلّص 75% على الأقل من الـ promise. الهدف 95%.

72. ليه صورة Express SLA فيها 6 breaches؟

الإجابة: هي من بروفة قصّرنا فيها الـ promise لدقيقتين وأجبرنا بعض orders الـ express تتأخر قصدًا، عشان نعرض breach و alert بتاعه لايف. دي مش نتيجة السيستم.

73. ليه events per minute بصفر في صورة الـ pipeline؟

الإجابة: الصورة اتاخدت في آخر بروفة 13 أكتوبر، بعد ما المولّد وقف. التايل بيعرض آخر دقيقة كاملة، وده بالظبط الحالة اللي "no events" alert معمول لاكتشافها.

74. ليه تايلات Gold update و publish برتقاني وبتعرض يوم واحد؟

الإجابة: في اللقطة دي آخر Gold update و publish كانوا من يوم، لأن مسار الـ batch بيشتغل عند الطلب مش باستمرار.

75. إيه الـ "hero scenario"؟

الإجابة: وضع `--scenario hero-stockout` (demo) بيخلّي تلات products "بطلة" في أزحم store تبدأ بـ 40 وحدة، وبيزوّد مشاهداتها وإضافاتها للـ cart، وبيعيد تخزينها بعد حوالي 90 ثانية من stock-out. نفس الـ seed بيدّي نفس الـ demo.

76. إيه اللي قستوه في الـ timeline بتاع الـ demo؟

الإجابة: يوم 12 أكتوبر أول product بطل وصل صفر بعد حوالي 4 دقايق من أمر الـ demo (Galabeya 61 في مدينة نصر، الساعة 21:16:14 بتوقيت القاهرة). alert الطلب على الخالص اشتغل بعدها بدقيقة، وإعادة التخزين حصلت بعد حوالي دقيقتين من stock-out. وبروفة express فيها 33 ،31 order express في الوقت و 2 متأخرين.

77. الـ live sink بيشتغل إزاي؟

الإجابة: بعد كل Silver batch، بيعيد حساب الدقايق وأزواج الـ store والـ product اللي الـ batch لمسها بس، وبيكتبها في Postgres. Grafana بيقراها بالـ `grafana_ro` role read-only.

78. إزاي بتتجنبوا الإنذارات الكاذبة من البيانات الفاضية؟

الإجابة: alerts النسب محتاجة حد أدنى للعدد (20 event أو صف على الأقل، 5 orders express على الأقل). البيانات الناقصة بتشغّل بس الـ alerts المعمولة للصمت. والـ dashboards بتعرض فجوات بدل صفر للنسب ومستويات الـ stock، وشرطة محايدة لما مفيش بيانات.

ح. Power BI والمقاييس

79. إزاي نموذج Power BI مبني؟

الإجابة: star schema في `Date` (Import mode: 4 dimensions و `Campaign` و `Store` و `Product`) والـ facts حواليهم. فيه 19 جدول و 27 relationship و 103 measure. مفيش علاقة many-to-many ولا فلتر ثنائي الاتجاه. بعد إعادة التصميم

في 13 أكتوبر بقى فيه 106 visual على 7 صفحات.

80. إيه الصفحات السبعة؟

الإجابة: Executive و Campaigns و Marketing and stock و Operations و Funnel و Products and cities و Customers. كل صفحة بتجاوب سؤال واحد وفيها 6 cards و 3 slicers بالكتير.

81. Power BI بيجيب بياناته منين؟

الإجابة: 11 جدول Gold من Databricks و 7 marts من Postgres، بـ credentials للقراءة فقط. الـ credentials مش بتتخزّن في الملف، فـ Power BI Desktop بيسأل عن Databricks token و Postgres password عند الفتح.

82. ليه Import mode مش DirectQuery؟

ببساطة: Import بيخلّي التقرير سريع ويشتغل من غير اتصال لايف وقت المناقشة.

الإجابة: الـ repository بيسجّل القرار (Import mode) من غير سبب مكتوب، فجاوب بالمنطق: Import mode بيخلّي التقرير سريع وبيشتغل من غير اتصال لايف وقت المناقشة، والبيانات بتتحدّث بعد Gold batch وبعد الـ builders بتوع Postgres. DirectQuery هيعتمد على إن Databricks warehouse يكون صاحي وهيبقى أبطأ.

83. ليه التقرير غيّر تصميمه؟

الإجابة: أعدنا تصميمه حوالين هوية واحدة: rail كحلي فيه الـ page navigator والـ slicers، و banner للعنوان بيقول سؤال الصفحة، وألوان Accessible Tidal، ومسافات ثابتة، وأعمدة مرتّبة بدل pie charts، وجداول مقروءة بعرض أعمدة صريح. التقرير بيتولّد من `scripts/make_pbip.py`، والاختبارات بتتأكد إن حقول كل visual موجودة في النموذج.

84. إيه هو "exposed spend" بالظبط؟

ببساطة: من budget الـ campaign، الجزء اللي غالبًا راح لناس نزلت على product خالص.

الإجابة: `exposed_spend = spend x oos_click_share`، حيث `oos_click_share` هي clicks الوصول على الـ out-of-stock products مقسومة على clicks الوصول اللي اتخدمت. الـ ad API بيدّي الـ spend لكل campaign لكل يوم بس، فبنفترض إن الـ spend موزّع بالتساوي على clicks الوصول. هو تقدير مش خسارة مقاسة، ومش "ضايع" بالكامل، لأن الزبون ممكن يشتري بديل.

85. إيه هو "revenue at risk"؟

الإجابة: إضافات للـ cart على الـ out-of-stock products مضروبة في السعر. في صفحة Marketing and stock هو 71,653 على بيانات محاكاة، والتلات products المعرّضة كلهم في store واحد، مدينة نصر.

86. ليه في ذروة يوم واحد في chart الـ revenue؟

الإجابة: البيانات المحاكاة فيها قفزة حجم يوم 9 أكتوبر بتشيل حوالي 55% من كل الـ revenue. صفحة Executive بتقول ده في العنوان الفرعي عشان محدش يقراه كاتجاه حقيقي.

87. إزاي بتتحسب نسب الـ abandonment؟

الإجابة: cart abandon rate هي `1 - orders_placed / cart_sessions`، و checkout abandon rate هي `1 - orders_placed / checkout_sessions`. بتبقى فاضية للشريحة الفاضية وبتتحصر بين 0 و 1، ومش بتعدّي 100%. في صفحة Funnel هما 69.1% و 30.7%، والـ session conversion هو 10.4%.

88. ليه التقرير بيقول "settling"؟

ببساطة: revenue التسليم بتاع الأيام الأخيرة لسه ممكن يزيد لأن orders لسه بتتسلّم.

الإجابة: الـ delivered revenue للأيام الأخيرة ممكن يزيد لسه كل ما orders تتسلّم، فـ `Maturity Label` بيعلّم الأيام بـ "Settling" أو "Settled". ده سبب إن الـ delivered ROAS الأخير بيبان واطي.

ط. الاختبار والأداء والاعتمادية

89. إزاي المشروع بيتختبر؟

الإجابة: على أربع مستويات: 1,169 اختبار على الـ host نجحوا بصفر فشل (13 أكتوبر)، و 217 اختبار Spark في 22 ملف نجحوا في Docker (12 أكتوبر)، و smoke test من 15 مرحلة، و benchmark و failure demo. الاختبارات بتغطي العقود والـ validator والـ simulator ومنطق Silver و Gold SQL وتعريفات الـ 66 check و Grafana JSON والـ alert rules ومولّد Power BI وحمايات الـ LLM.

90. ليه 218 اختبار متخطّين (skipped)؟

الإجابة: دي اختبارات Spark، وبتحتاج بيئة Docker وبتشتغل فيها ملف ملف. وفي اختبارات تانية بتتخطّى لما Postgres مش شغّال.

91. إيه نتيجة الـ benchmark؟

ببساطة: مليون event عدّوا على السيستم كله في حوالي 11 دقيقة على لاب توب واحد. ده قياس لـ setup واحد، مش promise بالسعة.

الإجابة: 1,000,000 event محاكى (1,022,333 رسالة Kafka) عدّوا على الـ pipeline كله في 654.73 ثانية على لاب توب واحد بـ 12 CPU و Docker محدود بحوالي 8 GB، باستخدام Databricks Free Edition SQL warehouse. ده حوالي 1,560 رسالة في الثانية end to end. هو بيقيس setup واحد ومش ادعاء سعة.

92. هل هيكبر (scale)؟

الإجابة: ما اختبرناش أكتر من الـ benchmark. نقط التصميم اللي بتساعد على التوسّع هي: topics مقسّمة، وجداول Delta مقسّمة، وكتابة idempotent، ومسار batch منفصل. التوسّع الحقيقي هيحتاج cluster و Kafka مُدار و object storage، وده ما بنيناهوش ولا قسناه.

93. إيه اللي أظهره الـ smoke test؟

الإجابة: أول تشغيل كامل له يوم 13 أكتوبر نجح في 10 مراحل وفشل في 3، ولا واحدة منهم كانت عيب في الـ pipeline: شرط بيانات غلط كان بيطلب جدول لحظي أثناء replay، و cast في بديل الاختبار لماكرو وقت Grafana، وصفوف قديمة في الـ scratch catalog. التلاتة اتصلّحوا. بعدها مراحل Gold نجحت في إعادة تشغيل يدوية. لسه مفتوح: تشغيل واحد أخضر من غير انقطاع يشمل مرحلة Spark.

94. إيه كان أصعب مشكلة؟

ببساطة: نخلّي كل خطوة آمنة لو اتعادت.

الإجابة: إننا نخلّي كل خطوة آمنة لو اتكررت. الـ pipeline لازم يستحمل crash و replay من غير تكرار، ويتعامل مع الـ late والـ out-of-order events، وبرضه يدّي أرقام متطابقة. الدليل هو فحص التكرار بعد الـ crash، وإعادة Bronze اللي سابت كل جداول Silver الـ 10 من غير تغيير.

95. إيه اللي بيحصل لو Databricks وقع؟

الإجابة: المسار اللحظي والـ lake المحلي بيفضلوا شغّالين لأنهم مش معتمدين عليه. وجداول Gold تقدر تتحمّل في Postgres من خلال الـ store backend التاني. Power BI هيعرض آخر بيانات اتعملها import.

ي. الأمان والحدود والمستقبل والعملية

96. إزاي بتتعاملوا مع الأسرار والصلاحيات؟

الإجابة: `.env` مش بيتعمله commit أبدًا، و `.env.example` فيه الأسماء بس. بورتات Kafka و Postgres و Grafana و Airflow مربوطة بـ Grafana . `127.0.0.1` و Power BI بيستخدموا credentials للقراءة فقط، والـ Databricks write token بيتستخدم في jobs التحميل بس. الـ smoke test بيرفض يلمس الـ catalog الحقيقي وبيشتغل في scratch catalog.

97. إيه أهم الحدود؟

الإجابة: كل البيانات محاكاة. علامة stock-out هي الـ stock وقت ما الدقيقة اتكتبت، فـ replay أو late event ممكن يعلّمها بشكل مختلف. الـ stock مش بيرجع عند الـ cancellation أو الإرجاع أو فشل الـ delivery. الـ exposed spend تقدير. أرقام الـ ads بيوم UTC والـ orders بيوم القاهرة. Gold فيه City بس ومفيش Store dimension. مؤشرات الـ express موجودة في جداول Postgres بس. والـ benchmark على setup واحد على Databricks Free Edition.

98. إيه اللي هتعملوه بعد كده؟

الإجابة: نحسب فترات الـ stock من Silver بوقت الـ event عشان علامة النفاد تبقى آمنة عند الـ replay، ونرجّع الـ stock عند الـ cancellation والإرجاع، وننشر جداول الـ stock على Databricks مع جدول Gold للطلب على الخالص، ونضيف delivery zones و Store dimension في Gold، ونملّى جدول التوصيات `rec_product` ، ونضيف الـ margin بعد تكلفة الكاش والـ cancellation، ونعمل اختبارات incrementality، ونخلّي جولات الـ simulator تتداخل عشان الترافيك اللحظي ميهدّاش، ونضيف Great Expectations أو dbt فوق فحوصات الـ SQL.

99. المساعد الـ LLM بيعمل إيه وإزاي بيتأمّن؟

ببساطة: الواحد بيسأل بالكلام العادي، والموديل بيكتب استعلام قراءة بس، ومبيشوفش بيانات شخصية.

الإجابة: بيخلّي المستخدم يسأل بلغة طبيعية، مثلًا أنهي campaign فيها أعلى CTR. الموديل بيكتب `SELECT` واحد في المرة بيشتغل على DuckDB views للقراءة فقط. الوصول للملفات مقفول، والاستعلامات ليها حد 20 ثانية و 200 صف، والنتايج بتتغلّف بـ `<query_result>` tags عشان البيانات متعدّيش كأنها تعليمات، والموديل مبياخدش أبدًا أسماء أو أرقام تليفونات، والـ API key مبيتسجّلش. الحمايات دي متختبرة.

100. المشروع كبير قد إيه، وإزاي اتبنى؟

الإجابة: حوالي 30,000 سطر Python (حوالي 17,800 منهم اختبارات) في أكتر من 100 commit، و 88 ملف اختبار. الشغل اتعمل على `funnelstream-build` branch مع توثيق وقاموس مقاييس و runbooks وملف handoff وقايمة تشييك قبل المناقشة في الـ repository. لو اتسألت مين عمل إيه، جاوب عن جزءك إنت وكن محدد.